Proyecto Seguridad Informática

Implementación de cifrado AES-128-CBC compatible con OpenSSL

Universitat Jaume I - EI1034. Curso 2025/2026
Izabela Andreea Slamnea - al439839@uji.es

INDEX

1. Introducción....3
2. Uso del software....4
    Instalación de requisitos....4
    Compatibilidad con OpenSSL....7
3. Implementación....9
4. Conclusión....13
5. Bibliografía....14

1. Introducción

En este proyecto se desarrolla una aplicación en Python llamada `xaes.py` cuya finalidad es cifrar y descifrar información utilizando criptografía simétrica.

La aplicación implementa el algoritmo AES de 128 bits en modo CBC (Cipher Block Chaining), utilizando PBKDF2 para derivar la clave criptográfica a partir de una contraseña proporcionada por el usuario. Además, se utiliza una sal aleatoria (salt) para aumentar la seguridad y garantizar la compatibilidad con OpenSSL.

El principal objetivo del proyecto consiste en desarrollar una herramienta totalmente compatible con el comando:

```
None
openssl aes-128-cbc -pbkdf2
```

de forma que:

- un archivo cifrado con OpenSSL puede descifrarse con `xaes.py`
- un archivo cifrado con `xaes.py` puede descifrarse con OpenSSL

El software puede ejecutarse en cualquier entorno compatible con Python 3, tanto en sistemas Windows como Linux o macOS, siempre que disponga de la librería criptográfica `PyCryptodome` y, opcionalmente, OpenSSL para realizar pruebas de compatibilidad.

El programa puede utilizarse desde:

- terminal CMD de Windows
- PowerShell
- Git Bash
- terminal Linux
- terminal integrada de distintos entornos de desarrollo

Durante el desarrollo de este proyecto se ha utilizado:

- Visual Studio Code (VSCode)

- terminal integrada en VSCode configurada en CMD de Windows
- Python 3
- librería criptográfica PyCryptodome.

2. Uso del software

Instalación de requisitos

Instalación y comprobación de Python

Para ejecutar el programa es necesario disponer de Python 3 instalado en el sistema.

La instalación puede comprobarse mediante:

```
None
python --version
```

Resultado esperado:

```
None
Python 3.x.x
```

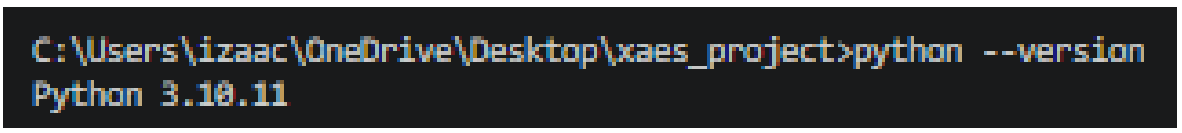


Figura 1. Comprobación de la instalación de Python desde la terminal integrada de VSCode.

Instalación de la librería PyCryptodome

La implementación utiliza la librería `PyCryptodome`, que proporciona las funciones criptográficas necesarias para AES y PBKDF2.

La instalación se realiza mediante:

```
None
pip install pycryptodome
```

Resultado esperado:

```
None
Requirement already satisfied: pycryptodome
```

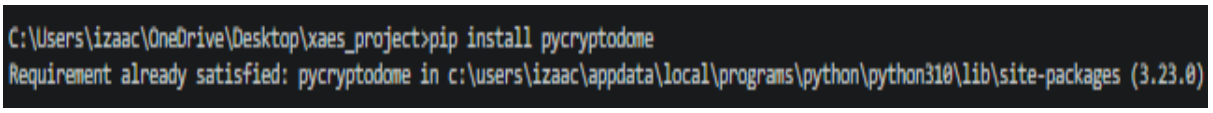


Figura 2. Instalación de la librería PyCryptodome.

Comprobación de OpenSSL

Para verificar la compatibilidad con OpenSSL es necesario disponer de OpenSSL instalado en el sistema.

La comprobación se realiza mediante:

```
None
openssl version
```

Resultado esperado:

```
None
OpenSSL 3.x.x
```

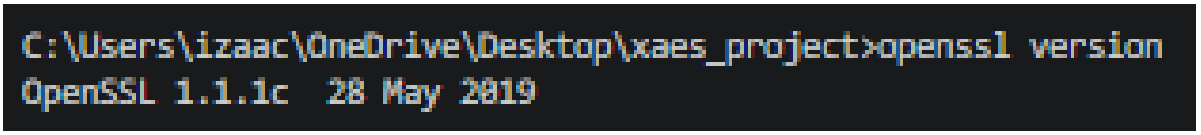


Figura 3. Verificación de la instalación de OpenSSL.

Funcionamiento del programa

El programa admite dos modos de funcionamiento:

- `-e` → cifrado

- `-d` → descifrado

La información se procesa utilizando:

- entrada estándar (`stdin`)
- salida estándar (`stdout`)

Esto permite utilizar redirecciones y tuberías desde la terminal, igual que OpenSSL.

Creación del archivo de prueba

Antes de realizar las pruebas se crea un archivo de texto:

```
None
echo Hola esto es una prueba AES CBC PBKDF2 > myfile.txt
```

Cifrado de archivos

Para cifrar un archivo se utiliza el siguiente comando:

```
None
type myfile.txt | python xaes.py -e clave123 > myfile.enc
```

Este comando:

1. Lee el contenido del archivo `myfile.txt`
2. Lo cifra utilizando AES-128-CBC
3. Guarda el resultado en `myfile.enc`

Resultado esperado (si no sale nada, está creado):

```
None
Archivo cifrado generado correctamente.
```

Descifrado de archivos

Se utiliza el archivo cifrado generado anteriormente.

```
None
type myfile.enc | python xaes.py -d clave123 > myfile.dec
```

Resultado esperado (si no sale nada, está creado):

```
None
Archivo descifrado generado correctamente
```

Verificación de integridad

```
None

fc myfile.txt myfile.dec
```

Resultado esperado:

```
None
FC: no differences encountered
```

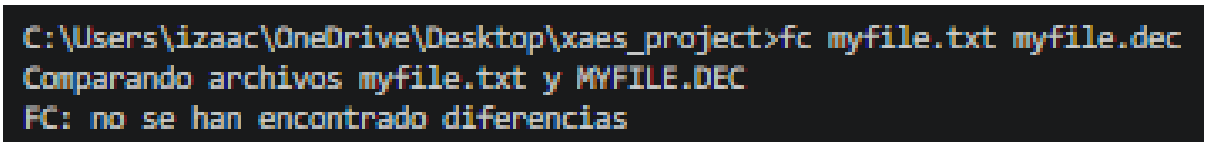


Figura 4. Comparación entre archivo original y archivo descifrado.

Compatibilidad con OpenSSL

Cifrado con OpenSSL → Descifrado con xaes.py

```
None
type myfile.txt | openssl aes-128-cbc -pbkdf2 -k clave123 > openssl.enc
type openssl.enc | python xaes.py -d clave123 > openssl.dec
```

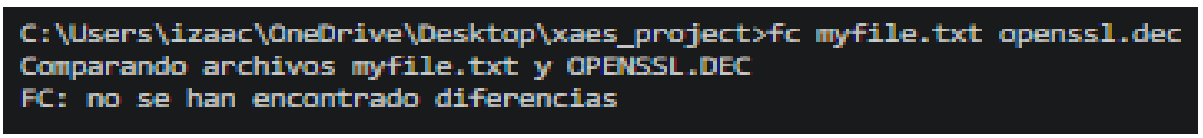


Figura 5. Compatibilidad OpenSSL → xaes.py.

Cifrado con xaes.py → Descifrado con OpenSSL

```
None
type myfile.txt | python xaes.py -e clave123 > python.enc
type python.enc | openssl aes-128-cbc -pbkdf2 -d -k clave123 > python.dec
```

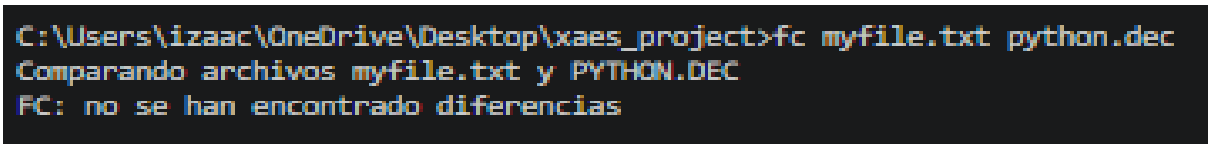


Figura 6. Compatibilidad xaes.py → OpenSSL.

Manejo de errores

```
None
type myfile.enc | python xaes.py -d claveIncorrecta
```

Resultado esperado:

```
None
error de contraseña incorrecta
```

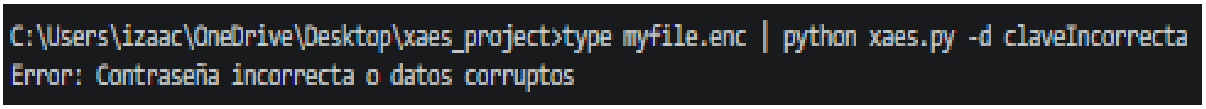


Figura 7. Manejo de errores ante contraseña incorrecta.

3. Implementación

Diseño general de la aplicación

La aplicación se ha diseñado como una herramienta de cifrado y descifrado de flujo, basada en procesamiento de datos a través de la entrada y salida estándar. Esto significa que el programa no trabaja directamente con ficheros, sino que recibe datos desde la entrada estándar (`stdin`) y devuelve el resultado por la salida estándar (`stdout`).

El objetivo del diseño es reproducir el comportamiento del comando de OpenSSL:

```
None
openssl aes-128-cbc -pbkdf2
```

de manera que exista compatibilidad total entre ambos sistemas en cualquier dirección de cifrado y descifrado.

El funcionamiento general del programa se puede entender como una tubería de procesamiento compuesta por cuatro etapas principales: lectura de datos, derivación de claves, procesamiento criptográfico y escritura del resultado.

Flujo de funcionamiento

El flujo interno de la aplicación sigue estos pasos:

1. El programa recibe los datos desde la entrada estándar.
2. Se determina si la operación es cifrado o descifrado mediante los parámetros de ejecución.
3. Se genera o se extrae la sal (salt) según el caso.
4. Se deriva la clave criptográfica y el vector de inicialización (IV).
5. Se realiza el proceso de cifrado o descifrado utilizando AES en modo CBC.
6. Se devuelve el resultado por la salida estándar.

Este diseño permite que el programa pueda integrarse fácilmente en tuberías de comandos, igual que herramientas clásicas de sistemas Unix.

Derivación de claves (PBKDF2)

Uno de los elementos más importantes de la implementación es la derivación de claves a partir de contraseñas.

Las contraseñas no se utilizan directamente como claves criptográficas, ya que esto sería inseguro. En su lugar, se utiliza PBKDF2 (Password-Based Key Derivation Function 2), un algoritmo diseñado específicamente para este propósito.

PBKDF2 realiza un proceso de transformación basado en:

- la contraseña introducida por el usuario
- una sal aleatoria (salt)
- un número elevado de iteraciones
- una función hash criptográfica (SHA-256)

El resultado de este proceso es una clave segura de 128 bits y un vector de inicialización (IV) de 128 bits.

Este mecanismo dificulta ataques por fuerza bruta, ya que cada intento requiere un coste computacional elevado.

Uso de salt

El sistema genera una sal aleatoria de 8 bytes en cada operación de cifrado. Esta sal es fundamental para garantizar que dos mensajes iguales cifrados con la misma contraseña produzcan resultados diferentes.

La sal se almacena junto con el mensaje cifrado en el propio archivo, siguiendo el formato estándar utilizado por OpenSSL:

```
None
Salted__ + salt + ciphertext
```

En el proceso de descifrado, esta sal se extrae del archivo y se utiliza nuevamente en la derivación de claves, garantizando que el resultado sea correcto.

Algoritmo de cifrado AES-128-CBC

El algoritmo utilizado es AES (Advanced Encryption Standard) con clave de 128 bits en modo CBC (Cipher Block Chaining).

El modo CBC funciona dividiendo los datos en bloques de tamaño fijo. Cada bloque de texto plano se combina con el bloque cifrado anterior antes de ser cifrado. Esto evita patrones repetitivos y aumenta la seguridad del sistema.

El primer bloque no tiene bloque anterior, por lo que se utiliza un vector de inicialización (IV), generado a partir del proceso de derivación de claves.

Padding de datos

AES requiere que los datos tengan un tamaño múltiplo de 16 bytes. Para solucionar este problema se utiliza padding PKCS#7.

Cuando el mensaje no cumple este requisito, se añaden bytes adicionales indicando el tamaño del relleno. Durante el descifrado, este padding se elimina automáticamente para recuperar el mensaje original.

Este mecanismo permite cifrar correctamente tanto texto como archivos binarios.

Formato de los datos cifrados

El resultado del cifrado sigue un formato compatible con OpenSSL. Este formato incluye:

- una cabecera fija (`Salted__`)
- la sal utilizada en la derivación de claves
- el texto cifrado resultante

Este diseño permite que OpenSSL pueda interpretar correctamente los archivos generados por el programa y viceversa.

Entrada y salida estándar

El programa está diseñado para trabajar exclusivamente con entrada y salida estándar.

Esto significa que:

- no se abren archivos directamente desde el código
- los datos pueden encadenarse mediante tuberías
- se mantiene compatibilidad con herramientas externas como OpenSSL

Este enfoque permite que el programa se comporte como una herramienta de sistema, en lugar de una aplicación tradicional basada en ficheros.

Manejo de errores

La implementación incluye mecanismos básicos de control de errores para garantizar la robustez del sistema.

En el proceso de descifrado se comprueba:

- la presencia de la cabecera `Salted__`
- la validez del formato del archivo
- la corrección de la contraseña
- la integridad del padding

Si alguna de estas condiciones falla, el programa muestra un mensaje de error y finaliza la ejecución de forma controlada.

Reproducibilidad del sistema

El diseño del sistema permite que otro programador pueda replicar la implementación sin necesidad de acceder al código fuente.

Esto es posible porque todos los componentes utilizados están basados en estándares conocidos:

- AES-128-CBC como algoritmo de cifrado
- PBKDF2 como función de derivación de claves
- formato compatible con OpenSSL
- uso de entrada/salida estándar

4. Conclusión

En este proyecto se ha desarrollado una herramienta de cifrado y descifrado en Python denominada `xaes.py`, basada en el algoritmo AES de 128 bits en modo CBC y el uso de PBKDF2 para la derivación de claves a partir de contraseñas. El objetivo principal ha sido implementar un sistema funcional que permita proteger información de forma segura y que, además, sea compatible con el comportamiento de OpenSSL utilizando la configuración `aes-128-cbc -pbkdf2`.

El sistema implementado garantiza la confidencialidad de los datos mediante el uso de cifrado simétrico AES-128-CBC, complementado con un proceso de derivación de claves seguro basado en PBKDF2. La incorporación de una sal aleatoria en cada operación de cifrado permite que el mismo mensaje cifrado con la misma contraseña genere resultados distintos, aumentando así la resistencia frente a ataques basados en patrones o tablas precomputadas.

Uno de los aspectos más importantes del proyecto es la compatibilidad total con OpenSSL, lo que permite realizar procesos de cifrado y descifrado en ambas direcciones sin pérdida de información. Esta interoperabilidad demuestra que tanto el formato de los datos cifrados como el proceso de generación de claves han sido implementados correctamente siguiendo estándares reales de criptografía.

En conclusión, el proyecto no solo cumple con los requisitos funcionales establecidos, sino que también permite comprender de forma práctica el funcionamiento interno de sistemas criptográficos modernos. Además, refuerza conceptos fundamentales como el cifrado simétrico, la derivación de claves, el uso de sal y la importancia de seguir estándares para garantizar la compatibilidad entre herramientas.

5. Bibliografía

Documentación oficial y técnica

- Python Software Foundation. *Python Documentation*. https://docs.python.org/3/
- PyCryptodome Team. *PyCryptodome Documentation*. https://pycryptodome.readthedocs.io/en/latest/
- OpenSSL Project. *OpenSSL Documentation*. https://www.openssl.org/docs/
- OpenSSL Project. *OpenSSL enc manual (AES y PBKDF2)*. https://docs.openssl.org/3.3/man1/openssl-enc/

Estándares criptográficos

- National Institute of Standards and Technology (NIST). *FIPS PUB 197: Advanced Encryption Standard (AES)*. https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf
- RFC 8018. *PKCS #5: Password-Based Cryptography Specification (PBKDF2)*. https://datatracker.ietf.org/doc/html/rfc8018

Material docente del proyecto

- Universitat Jaume I (UJI). *Práctica 2 – OpenSSL*. https://aulavirtual.uji.es/pluginfile.php/7467760/mod_resource/content/23/Practica2-openssl.pdf
- Universitat Jaume I (UJI). *Teoría 2 – Criptografía*. https://aulavirtual.uji.es/pluginfile.php/7467753/mod_resource/content/19/teoria_2_criptografia.pdf

---

Recursos complementarios

- Kolin Stürt. *AES Encryption using OpenSSL*. https://kolinsturt.github.io/lessons/2011/11/15/aes-encryption-using-openssl/
- Saraiva, P. M. *OpenSSL acceleration using GPUs*. https://hgpu.org/?p=11278